

Contents

1	Introduction to Python	3
2	Roles in Analytics	3
3	Types of Analytics	3
4	Jupyter Notebook	3
4.1	Jupyter Notebook State	3
4.1.1	Edit State	3
4.1.2	Select State	3
4.2	Jupyter Notebook Modes	4
5	Comments	4
5.1	Single-Line Comment	4
5.2	Multi-Line String	4
6	Variable	4
7	Built-in Date Types	4
7.1	Checking Data Types	4
7.2	Implicit Type Conversion	5
8	Print	5
9	Operators	5
9.1	Arithmetic Operators	5
9.2	Arithmetic Assignment Operators	5
9.3	Logical Operators	5
9.4	Comparison Operators	5
9.5	Questions	5
10	Conditional Statements	6
10.1	Type 1: if	6
10.2	Type 2: if-else	6
10.3	Type 3: if-elif-else	6
10.4	Type 4: Ternary Conditional Expression	6
10.5	Question	6
11	Keywords	7
12	Input()	7
13	int()	7
14	Loops	7
15	Data Structures	8
15.1	Tuples:	8
15.1.1	Tuple Indexing:	8
15.1.2	Tuple Slicing:	8
15.2	List:	9
15.2.1	Update Operations	9
15.2.2	Deletion Operations	9
15.2.3	Miscellaneous Operations	10
15.3	Dictionary(key-value pair)	10

16 Sets	11
16.1 Set Operations	11
16.1.1 Union of Two Sets	11
16.1.2 Intersection of Two Sets	11
16.1.3 Difference Operation	11
16.1.4 Symmetric Difference	11
17 List Comprehension	11
18 User-Defined Functions(UDF)	11
19 Variable-Length Positional Arguments (*args)	12
20 Variable-Length Keyboard Arguments(**kwargs)	12
21 Lambda Functions	13

1 Introduction to Python

Python is a high-level interpreted, open-source programming language that is widely used for software development, automation, data analysis, artificial intelligence, machine learning, web development, scientific computing, and many other domains.

2 Roles in Analytics

According to the uploaded material, common analytics role include:

- Data Engineer
- Data Analyst/ Business Analyst
- Data Scientist/ Decision Scientist
- Machine Learning/ Artificial Intelligence Engineer
- MLOps (Machine Learning Operations)

3 Types of Analytics

- Descriptive Analytics
- Diagnostic Analytics
- Inferential Analytics
- Predictive Analytics
- Prescriptive Analytics
- Cognitive Analytics

4 Jupyter Notebook

Jupyter Notebook is an interactive development environment that allows users to combine executable Python code, formatted text, mathematical equations, tables, and visualizations in a single document.

4.1 Jupyter Notebook State

Every notebook cell has two states.

4.1.1 Edit State

Used for editing the contents of a cell.

Common shortcuts:

- Ctrl+Enter: Execute current cell and remain in the same cell.
- Shift+Enter: Execute current cell and move to the next cell.

4.1.2 Select State

Used for notebook management.

Important shortcuts:

- ESC+C: Copy cell
- ESC+X: Cut cell
- ESC+V: Paste cell

- ESC+A: Insert cell above
- ESC+B: Insert cell below
- ESC+DD: Delete current cell

4.2 Jupyter Notebook Modes

Three notebook modes are introduced.

- **Code Mode:** Used for writing executable Python code.
Shortcut: ESC+Y
- **RawNBConvert Mode:** Behaves like plain text without formatting.
Shortcut: ESC+R
- **Markdown Mode:** Used to create documentation.
Shortcut: ESC+M

5 Comments

Comments are not executed by the Python interpreter. Primary uses include:

- Documentation
- Recording observations.

5.1 Single-Line Comment

Begins with the symbol #

5.2 Multi-Line String

Python does not have true multi-line comments.

Instead, triple-quoted strings are commonly used for documentation (docstrings).

The uploaded code is reproduced exactly below.

```
# This is the comment
```

```
""" This is
multi-line string"""
```

6 Variable

A variable acts as a container that stores a value in memory.

7 Built-in Data Types

Data Type	Description	Conversion Function
int	Whole numbers	int()
float	Decimal numbers	float()
str	Sequence of characters	str()
bool	Logical values	bool()

7.1 Checking Data Types

type(a)-Returns the type of the object.

7.2 Implicit Type Conversion

```
a=20.2
b=10
c=a+b # Implicit conversion of b to float type
type(c)-float
```

8 Print

The function print() prints its argument but returns None.

9 Operators

9.1 Arithmetic Operators

- +
- -
- *
- /
- %
- //

9.2 Arithmetic Assignment Operators

- +=
- -=
- *=
- /=

9.3 Logical Operators

- and
- or
- not

9.4 Comparison Operators

- >
- <
- >=
- <=
- ==
- !=
- in

9.5 Questions

- Identify if the given number is positive (true) or negative (false).

10 Conditional Statements

Conditional Statements allow a program to make decisions based on one or more logical conditions. They control the execution flow by executing blocks of code depending on whether a condition evaluates or True or False.

Types of Conditional Statements

10.1 Type 1: if

if condition:

... statement

The block executes only when the condition is True.

10.2 Type 2: if-else

if condition:

... statement

else:

... statement

Exactly one block executes.

10.3 Type 3: if-elif-else

if condition:

... statement

elif condition:

... statement

...

else:

... statement

Python evaluates the conditions sequentially until one becomes True.

10.4 Type 4: Ternary Conditional Expression

A ternary expression performs conditional evaluation in a single line.

value_if_true if conditions else value_if_false

10.5 Question

- Identify if the given number is even or odd.
- Identify if the given number is
positive even,
negative even
positive odd and
negative odd number
- Given a value, identify if it's numeric or non-numeric type.
- Take a number as an input, and annotate:
'foo' if the number is divisible by 3
'buzz' if the number is divisible by 5
'foobuzz' if the number is divisible by 3 and 5
print the number a such if not divisible by either 3 or 5.
- Identify if a given number is positive or negative with ternary.
- Identify if a given number is positive or negative or 0 with ternary.
- Identify if the given number is even or odd with ternary.

11 Keywords

- try- Contains statements that may raise an exception.
- except- Executes when an exception occurs.
- finally- Executes regardless of whether an exception occurred.

12 Input()

Syntax: input(prompt)

Returns user input as a string.

13 int()

Convert a string into an integer.

Syntax: int(value)

14 Loops

Loops are control flow statements that allow a block of code to execute repeatedly. They are one of the most important concepts in Python because they automate repetitive tasks and reduce manual coding. Python provides two primary looping constructs:

- for Loop (Range-based Loop)
Syntax:
for variable in iterable:
... statements
- while Loop (Entry Controlled Loop) Syntax:
cnt=0
while condition:
... statements ... cnt+=1 or cnt-=1
- **Important Notes**
 - A while loop can easily become an infinite loop if the stopping condition is not updated correctly.
 - A for loop is generally used when the number of iterations is known.
 - A while loop is suitable when the number of iterations is unknown.

Problems

- Write the table of 5 (for loop)
- Write the table of 5 in inverse manner (for loop)
- Write the table of 5 (while loop)
- write the table of 5 in inverse manner (while loop)
- From a given set of number starting from 1 to 100, identify the sum of even and odd numbers.
- Identify the number of digits in a given number.
- Identify the sum of digits in a given number.

15 Data Structures

Data Structures are specialized containers used to store and organize multiple values efficiently. Try enable efficient **CRUD**(Create, Read, Update, Delete) operations and are fundamental in Python programming, data analytics, and software development.

Python provides several built-in data structures:

15.1 Tuples:

A tuple is an ordered, immutable collection of objects.

Once a tuple is created, its elements cannot be modified.

Syntax: `t1=(1,2,3)`

Characteristics

- Ordered collection.
- Immutable.
- Allows duplicate values.
- Can store heterogeneous data.
- Supports indexing and slicing.
- Faster than lists for read-only data.

Problems

- From a given tuple identify all the numeric values and print out the final sum.

```
t1=(1,2,3)
```

```
t2=(3,4,5.6,'abc',True, False, t1)
```

15.1.1 Tuple Indexing:

A tuple supports zero-based indexing.

15.1.2 Tuple Slicing:

The notebook introduces slicing using the following syntax.

Slicing/Dicing in tuple

Syntax: `[[start]:[end]:[step]]`

- start- starting index
- end- ending index(excluded)
- step- increment/decrement

Rules:

- The starting index is included.
- The ending index is excluded.
- Default step is 1.
- A negative step traverses the tuple in reverse order.

Example:

- `t2[:3]` Output- (3,4,5.6)
- `t2[4:6]` Output- (True, False)

Problems

- Fetch alternative numbers start from 1
t3=tuple(range(1,10))
- Fetch only even numbers from t3
- Find common elements in the given tuples.
t_x = (1,2,3,4,5)
t_y = (4,5,6,7)

15.2 List:

A list is an ordered, mutable collection that can store elements of different data types.

A Python list stores multiple values inside square brackets.

Syntax:

```
variable_name = [item1, item2, item3]
```

Characteristics

- Ordered
- Mutable
- Duplicates Allowed
- Indexed
- Heterogenous Data
- Resizable

Python provides the build-in function dir() to display all available attributes and methods.

The notebook categorizes the operations as follows.

15.2.1 Update Operations

- **append() Method:** The append() method adds exactly one element to the end of a list.
Syntax: list_name.append(element)
Time Complexity: O(1)
- **extend() Method:** The extend() method inserts multiple elements into an existing list.
Syntax: list_name.extend(iterable)
Time Complexity: O(k), where k is the number of inserted elements.
- **insert() Method:** The insert() method inserts an element at a specified position.
Syntax: list_name.insert(index,value)
Time Complexity: O(n)

Problems

- Identify the even and odd numbers in the given list and separate them in 2 different list.
lst=list(range(101,201,1)).
- Identify the even and odd numbers in the given list and capture the differences in average of even_list and average of odd_list
lst=list(range(1,101,1)).

15.2.2 Deletion Operations

- **remove() Method:** The remove() method deletes the first matching value.
Syntax: list_name.remove(value)
Time Complexity: O(n)
- **pop() Method:** The pop() method removes an element from a list and returns the removed element.
Syntax: list_name.pop([index]). If no index is supplied , the last element is removed.
Time Complexity: O(1) or O(n) if indexed is given.

- **clear() Method:** The clear() method removes every element from the list.
Syntax: list_name.clear()
Time Complexity: O(n)

Problems

- Identify the count of all the data types in the given list.
lst=[1,2,3,5.3,3.2,2.1,True, False, 'abc', 'xyz']
- Perform element wise addition in 2 different list and output in a separate list.
l1=[1,2,'abc',3]
l2=[4,5,6,'def']

15.2.3 Miscellaneous Operations

- **count() Method:** Counts the number of occurrences of a specified value.
Syntax: list_name.count(value)
Time Complexity: O(n)
- **index() Method:** Returns the index of the first matching element.
Syntax: list_name.index(value)
Time Complexity: O(n)
- **sort() Method:** The sort() method arranges the elements of a list in ascending order by default.
Syntax: list_name.sort()
Time Complexity: O(nlogn)
- **reverse() Method:** Reverse the existing order of elements. It does not perform sorting.
Syntax: list_name.reverse()
Time Complexity: O(n)

Problems

- Fetch the index of second 'abc' (duplicate) in l1=[1,2,'abc',3].
- Count the occurrence of duplicates in the given list, lst=[1,2,3,2,3,1,4,5,6].

15.3 Dictionary(key-value pair)

- A dictionary is a mutable collection of key-value.
- Concept of indexes don't exist.
- Keys are unique in dictionary; Keys are only of immutable type: int, float, str, bool, tuple.
- Value can be anything

Syntax: d={key1:value1, key2:value2}

Problems

- Find common elements in the given tuples. Hint: Use Dictionary
t_x=(1,2,3,4,5)
t_y=(4,5,6,7)
- Count the occurrence of duplicates in the given list. Hint: Use Dictionary
lst=[1,2,3,2,3,1,4,5,6]
- Identify the count of all the data types in the given list. Hint: Use Dictionary
lst=[1,2,3,5.3,3.2,2.1,True,False,'abc','xyz']
- Find uncommon elements in the given tuples. Hint : Use Dictionary
t_x=(1,2,3,4,5)
t_y=(4,5,6,7)

16 Sets

Note: We avoid using sets due to following properties.

Properties:

1.Sorting the elements 2.Deleting the duplicates

s1={1,2,3,4,3,2,1,4,5,6,7} s1

Output:

1,2,3,4,5,6,7

The dir() function displays all available methods supported by the set object.

16.1 Set Operations

16.1.1 Union of Two Sets

The union operation returns every unique element that exists in either of the two sets.

Purpose:Combine two sets while automatically removing duplicates values.

Syntax: set1.union(set2)

16.1.2 Intersection of Two Sets

The intersection operation returns only the common elements.

Syntax: s1.intersection(s2)

16.1.3 Difference Operation

Difference returns elements that belong to one set but not the other. **Syntax:**s1.difference(s2)-Elements from s1 except common elements

s2.difference(s1)-Elements from s2 except common elements

16.1.4 Symmetric Difference

Symmetric difference returns elements that appear in exactly one of the two sets.

Syntax: s1.symmetric_difference(s2)

17 List Comprehension

A List Comprehension creates a new list by evaluating an expression for every element in an iterable.

Instead of writing multiple lines using a for loop and append(),Python allows both operations to be written in a single statement.

Syntax: new_list= [expression for item in iterable]

syntax with condition: new_list = [expression for item in iterable if condition]

Problems

- Generate a list containing only even numbers between 1 and 100 by list comprehensive.
- Fetch Only even numbers after adding 100 to values<25 and 200 to values>=25. Hint: Use List Comprehensive

18 User-Defined Functions(UDF)

Python provides tow broad categories of functions:

– **Built-in (System-defined) Functions:**

* sum()

* max()

* min()

- * len()
- * print()
- * type()
- * abs ()
- * round()
- * sorted()

- **User-Defined Functions:** A User-Defined Function (UDF) is a function created by the programmer to perform a specific task. Function improve code reusability, readability and maintainability.

Syntax:

```
def function_name(parameters):
. statements
. return value
```

Problems

- * Write a UDF to add 2 elements. Perform proper data validation.
- * Write a UDF to perform arithmetic operations on 2 elements.

Python passes object references.

- Immutable Objects include
 - Integers
 - Floats
 - Strings
 - Tuples
- Mutable objects include
 - Lists
 - Dictionaries
 - Sets

19 Variable-Length Positional Arguments (*args)

Python allows a function to accept a variable number of positional arguments using the special syntax ***args-Stores all positional arguments inside a tuple.**

Instead of defining a fixed number of parameters, the function can receive zero, one ,or many positional arguments.

Problems

- Provide sum of n numbers with proper date validation. Hint: Use *args
- Write a UDF to perform element wise addition in 2 different list and output in a separate list.
 l1=[1,2,'abc',3,5]
 l2=[4,5,6,'def']
- Write a UDF to identify the missing number from a given set of n sequential numbers:
 1,2,3,4,6,7,8,9,10 - 5

20 Variable-Length Keyboard Arguments(**kwargs)

**kwargs allows a function to accept any number of keyword arguments.

Unlike *args, which stores positional arguments inside a tuple, **kwargs stores keyword arguments inside a dictionary. ****kwargs is always a dictionary.**

21 Lambda Functions

A lambda function is an anonymous function that consists of a single expression.

Unlike normal functions, it doesnot require the def keyboard.

Syntax: lambda parameters: expression

Comparision

- Normal function
def square(x): . return x*x
- lambda x:x*x